VNG: A Markov-Chain-Based Dynamic Graph Convolution Network

Jialing Bi Yanping Zheng Zhifei Li Zhewei Wei*
Renmin University of China, Beijing, China, 100872
{bijialing22, zhengyanping, ZhifeiLi, zhewei}@ruc.edu.cn

Abstract

Decoupled Graph Convolutional Networks (GCNs), which decouples the feature transformation and information propagation stages for node representation learning, have achieved great success and become the latest paradigm of GCNs. However, a significant limitation of current decoupled GCNs is their focus on fixed and static graphs. While some recent approaches have attempted to address evolving graphs, they mainly focus on edge-level dynamics and assume static node numbers and features, limiting their applicability to real-world data. In this paper, we introduce Variable Node GCN (VNG), a novel framework capable of addressing both edge-level and node-level dynamics. Our core insight is that the iterative computation of information propagation in decoupled GCNs can be reformulated as a homogeneous first-order Markov chain through specific mathematical transformations. Building on this, we employ iterative aggregation techniques for decoupling Markov chains to effectively update node representations in evolving graph structures. This approach provides a flexible solution for both fixed-node and variable-node evolving problems. We present formal proofs and experimental results demonstrating that VNG is both efficient and effective, outperforming state-of-the-art baselines across various datasets.

1. Introduction

Graphs can represent objects and their complex interactions in real-world systems, such as social networks, e-commerce platforms, and molecular structures. Graph machine learning can reveal hidden patterns and insights by analyzing graph data, ultimately advancing the capabilities of various applications. For example, in recommender systems, the analysis of interaction graphs between users and items enables a more precise comprehension of users' preferences, which improves the quality of recommendations [14]. Graph Neural Networks (GNNs) effectively model the intricate relationships within graphs and have become a widely used graph analysis paradigm due to their exceptional performance [9, 16, 18].

Graph Convolutional Networks (GCNs) are a type of GNN designed to apply convolutional operations to graph data, extending the concept of convolution from the image domain to the graph domain [16]. Because graph structures are irregular compared to the fixed structure of images, various graph convolution techniques have been developed to address this complexity. These methods can be broadly classified into two categories: spectral convolutions [3, 7, 16], which leverage the graph Fourier transform to map node representations into the spectral domain, and spatial convolutions that operate on local node neighborhoods. According to the analysis in [1], spatial convolutions are generally more training-efficient than their spectral counterparts. Notably, when a polynomial spectral kernel is used, spectral convolutions can effectively be simplified to spatial convolutions. Therefore, GCNs that utilize spatial convolutions are essential for many practical applications and merit thorough investigation.

The original design of Spatial GCNs features a coupled approach, where the feature transformation and information propagation stages are intertwined and inseparable [3, 4, 7, 13, 16, 26]. While this approach has been effective in achieving good performance, it also comes with notable limitations, such as reduced flexibility, increased risk of over-fitting, and inefficiency in capturing deeper structural information from the graph. A series of works introduce Decoupled GCNs to address these challenges by separating the feature transformation and information propagation stages, improving modularity and efficiency. The feature transformation stage employs non-linear functions to generate node representations for subsequent graph processing or downstream tasks. In the propagation stage, each node aggregates data from its neighbors while simultaneously sending information to them. This decoupling allows for more effective modeling of the structural dependencies inherent in graphs. Methods such as SGC [27] and APPNP [11] have demonstrated that they can enhance training efficiency and alleviate over-fitting. Furthermore, the decoupled architecture allows independent optimization and integration of various transformation and propagation mechanisms.

However, existing methods are generally designed for static graphs. While a few approaches have been proposed to handle dynamic graphs, they primarily focus on edge and feature updates, presuming that the set of nodes remains fixed [10, 32, 33]. This assumption is unrealistic, as in social networks, users can join and leave at any time. Addressing the variable-node evolution problem, where the number of nodes fluctuates with evolving edges, is particularly challenging because it involves managing the addition and removal of nodes, and ensuring the model can continuously learn and adapt to the changing graph topology.

We notice that the iterative computation of node representations in decoupled GCNs can be reformulated as a homogeneous first-order Markov chain. Building on this insight, we introduce Variable Node GCN (VNG), a method designed to simultaneously address both node and edge evolution in dynamic graphs. Our contributions are summarized as follows:

- We propose Variable Node GCN (VNG), a novel technique that reformulates the iterative computation of node representations into a Markov chain framework, enabling efficient updates and precise management of evolving nodes and edges.
- We develop a Markov-chain-based algorithm to update node representations for dynamic evolving graphs, with a computation process and results that are theoretically well-founded. Our approach efficiently handles evolving problems in both fixed-node and variable-node scenarios and incrementally updates node representations based on prior results, avoiding needless recomputation while preserving accuracy.
- We conduct extensive experiments and ablation studies to validate the efficiency and effectiveness of our method on node classification tasks in dynamic graphs, benchmarking it against baseline algorithms. The results demonstrate that our approach effectively handles complex graph evolution scenarios and delivers superior performance.

2. Preliminary

Consider a graph denoted as $G = (V, E)$, where n denotes the number of nodes in V, and m represents the number of edges in E. Generally, nodes represent entities of interest, and edges are the relationships between these entities. Let $X \in \mathcal{R}^{n\times f}$ denote the feature matrix of nodes, where f represents the number of features per node. Let $Y \in \mathcal{R}^{n\times c}$ denote the class (or label) matrix where c represents the number of classes. $A \in \mathcal{R}^{n\times n}$ is the adjacency matrix of the graph G, and $\tilde{A} = A + I_n$ is the adjacency matrix with added self-loops. Additionally, $A^\top$ is the transpose of matrix A and A^{-1} is the inverse of matrix A. $D \in \mathcal{R}^{n\times n}$ is the diagonal degree matrix with $D(i,i) = \Sigma_j A(i,j)$, and $\tilde{D} = D + I_n$. We define N_u as the neighbor set of node u, and we have $\tilde{N}_u = N_u \cup \{u\}$. Detailed notations are shown in the Appendix.

2.1. Graph Convolutional Networks (GCNs)

Analogous to Multi-Layer Perceptrons (MLPs), GCNs input graph feature matrix X and learn node representations across multiple layers. These representations are then used for training models on downstream tasks, such as node classification. A L-layer GCN learns node representations from both the graph structure and the feature matrix X, with each node aggregating information from its neighbors at each layer. At each graph convolution layer, the hidden node representations are updated as follows:

$$h_i^{(\ell)} = \sigma\left(\sum_{j\in\tilde{N}_i} \frac{1}{\tilde{D}(i,i)} h_j^{(\ell-1)}\Theta^{(\ell-1)}\right), \ell = 1, \cdots, L \tag{1}$$

which can be written equivalently in a matrix form as follows:

$$H^{(\ell)} = \sigma\left(\tilde{A}\tilde{D}^{-1}H^{(\ell-1)}\Theta^{(\ell-1)}\right), \ell = 1, \cdots, L \tag{2}$$

where $H^{(\ell)}$ is the hidden node representations for the ℓ-th layer, and $H^{(0)}$ refers to the initial graph feature matrix X, $\sigma(\cdot)$ is the non-linear activation function, such as ReLU. Θ^ℓ is a trainable weight matrix at the ℓ-th layer to perform the linear transformation.

Consider $\hat{\tilde{A}} = \tilde{A}\tilde{D}^{-1}$ as the propagation matrix. From Equation (2), we observe that each layer-wise graph convolution consists of three operations:

- *Feature Transformation* $B = H^{(\ell-1)}\Theta^{(\ell-1)}$, which applies a linear transformation to the features;
- *Neighborhood Aggregation* $C = \hat{\tilde{A}}B$, which propagates information from neighboring nodes;
- *Nonlinear Activation* $H^{(\ell)} = \sigma(C)$, which introduces nonlinearity to the updated node representations, enabling the model to capture complex relationships in the graph.

Based on matrix multiplication properties, the order of the feature transformation and neighborhood aggregation operations can be interchanged, with the nonlinear activation always executed last. Since each layer in a vanilla GCN needs to learn a corresponding parameter matrix and apply a nonlinear activation function, the feature transformation and neighborhood aggregation operations remain coupled and cannot be separated. As a result, full-batch training is required, making it difficult to scale to large datasets [27].

2.2. Decoupled GCNs (DGCNs)

Some studies [19, 29] have noted that the depth of coupled GCNs is often quite limited, with 2 to 3 layers typically being optimal. This limitation arises for two main reasons. First, GCNs with excessive layers possess more parameters, which complicates the training process. Second, using too many layers can result in over-smoothing due to the averaging effect of aggregation, where nodes at deeper layers converge to identical representations.

Linear DGCNs. SGC [27] addresses the train efficiency problem of coupled GCNs by removing the nonlinearities between GCN layers. Specifically, it converts the nonlinear structure of coupled GCNs:

$$\mathrm{H}^{(\ell)} = \sigma(\ldots\sigma(\hat{\hat{\mathrm{A}}}\mathrm{X}\Theta^{(0)})\ldots)\Theta^{(\ell-1)} \tag{3}$$

into a linear form:

$$\mathrm{H}^{(\ell)} = \hat{\hat{\mathrm{A}}}\ldots\hat{\hat{\mathrm{A}}}\mathrm{X}\Theta^{(0)}\ldots\Theta^{(\ell-1)} \tag{4}$$

By re-parameterizing the weights in $\Theta^{(0)}\ldots\Theta^{(\ell-1)}$ into a single matrix Θ and raising $\hat{\hat{\mathrm{A}}}$ to the ℓ-th power (i.e., $\hat{\hat{\mathrm{A}}}^{\ell}$), Equation (4) can be further simplified as:

$$\mathrm{H}^{(\ell)} = \hat{\hat{\mathrm{A}}}^{\ell}\mathrm{X}\Theta \tag{5}$$

which suggests a natural decoupled implementation method for GCNs as:

$$\mathrm{H}^{(0)} = \mathrm{X}\Theta \tag{6}$$

$$\mathrm{H}^{(\ell)} = \hat{\hat{\mathrm{A}}}\mathrm{H}^{(\ell-1)} \tag{7}$$

in which, Equation (6) represents the feature transformation operation, while Equation (7) denotes the neighborhood aggregation operation. It is important to note that since weights are not required during the neighborhood aggregation operation, the value of ℓ can be sufficiently large, potentially approaching infinity. This allows for the complete utilization of the entire graph structure.

SGC demonstrates performance that is comparable to, or even surpasses, traditional coupled GCNs across various tasks, while also being more computationally efficient and requiring significantly fewer parameters. However, when ℓ is large, the challenge of over-smoothing due to averaging-based neighborhood aggregation persists, resulting in a loss of focus on local neighborhoods.

PPR-based DGCNs. To further remedy the over-smoothing problem, inspired by Personalized PageRank (PPR), PPNP and APPNP [11] improves the decoupled GCN by adding a teleport part to $\mathrm{H}^{(0)}$:

$$\mathrm{H}^{(\ell)} = (1-\alpha)\hat{\hat{\mathrm{A}}}\mathrm{H}^{(\ell-1)} + \alpha\mathrm{H}^{(0)} \tag{8}$$

where $\alpha \in (0,1]$ is a hyper-parameter of teleport probability and $H^{(0)}$ is the hidden node feature matrix obtained from the input node feature matrix X through the model-agnostic neural network f_Θ, specifically $H^{(0)} = f_\Theta(X)$. It is important to note that: 1) the introduction of the teleport part helps preserve the node's local neighborhood even as ℓ approaches infinity; 2) in Equation (8), all parameters are encapsulated within $f_\Theta(.)$, while the computation of the matrix $P = (1-\alpha)\hat{\hat{\mathrm{A}}}$ depends solely on the graph structure and is independent of $f_\Theta(.)$.

Due to their efficiency and flexibility, PPR-based DGCNs have emerged as a leading paradigm in GCNs. Consequently, the subsequent discussion in this paper primarily focuses on PPR-based DGCNs.

2.3. Dynamic Graphs

An evolving graph models the dynamism of a graph and captures its changes in structure and features over time. Given an initial graph $G_0 = (V_0, E_0)$ with feature matrix X_0, an evolving graph describes a dynamic graph evolution process formulated by a sequence of actions action$_1$, action$_2$, ..., action$_k$, where k is the number of graph actions. Specifically, action$_i$ denotes an operation conducted on G_0 at time i. The types of action$_i$ can be any of the following: Node$_{insert}$, Node$_{delete}$, Edge$_{insert}$, Edge$_{delete}$, Feature$_{update}$. We assume the graph evolves starting from time $t = 0$, and multiple actions can be performed on the graph at each time step. More precisely, let G_t denote the graph at the time step t. Then G_t can be derived from G_0 by executing a series of actions (i.e., action$_1$, action$_2$, ..., action$_k$).

Problem Definition. Given a graph $G_{t-1} = (V_{t-1}, E_{t-1})$ with a feature matrix X_{t-1} and corresponding node representations H_{t-1} at time step t-1, and given a new graph G_t that evolves from G_{t-1} at time step t through a series of graph actions, our goal is to build a dynamic decoupled GCN model which can compute the updated node representation H_t for G_t using the components of H_{t-1} with less effort than computing H_t from scratch.

When the evolution of a graph does not involve an increase or decrease in the number of nodes, it is termed a fixed-node evolving problem. Conversely, when new nodes are added or existing nodes are removed, it is referred to as a variable-node evolving problem. The variable-node evolving problem is inherently more challenging, with the fixed-node evolving problem representing a special case within it. Our objective is to develop a general algorithm capable of effectively handling node representation updates for both types of problems.

3. Method

To maintain the timeliness of node embeddings in dynamic graphs, a simple approach is to recompute node representations after each graph update. However, this method is computationally expensive and inefficient, particularly when handling large-scale data. Additionally, since most updates typically affect only a subset of nodes, it leads to redundant recalculations, wasting computational resources. In this section, we propose an incremental approach to computing node representations that improves efficiency and reduces computational costs. First, we illustrate that using specific mathematical operations, the layer-wise convolution computation of decoupled GCNs can be converted to a homogeneous first-order Markov chain, as detailed in Section 3.1. We then utilize iterative aggregation methods to decouple the Markov chains, enabling the efficient updating of node representations in dynamic graphs. Based on this framework, we introduce the VNG algorithm, which addresses both fixed-node and variable-node evolving issues, as elaborated in Section 3.2. Section 3.3 presents a theoretical justification and analysis for the computational process and results.

3.1. PPR-based DGCN vs. Markov Chain

By splitting the node representation matrix from Equation (8) into f vectors of dimension n, the computation of the decoupled GCN at the ℓ-th layer can be expressed in the following equivalent vector form:

$$H^{(\ell)}(:,i) = (1-\alpha)\hat{\tilde{\mathrm{A}}}H^{(\ell-1)}(:,i) + \alpha H^{(0)}(:,i). \tag{9}$$

For simplicity, let $\pi = H^{(\ell)}(:,i)$ and $\mathbf{r} = H^{(0)}(:,i)$. Substituting these into Equation (9) and assuming $\ell \to \infty$, we have:

$$\pi^\top = (1-\alpha)\pi^\top\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{r}^\top \tag{10}$$

Let $\mathbf{e}$ denote an n-dimensional column vector of ones. Therefore, we have $\pi^\top\mathbf{e} = 1$, and Equation (10) can be reformulated as follows:

$$\pi^\top = (1-\alpha)\pi^\top\hat{\tilde{\mathrm{A}}}^\top + \alpha\pi^\top\mathbf{e}\mathbf{r}^\top. \tag{11}$$

This expression can be further simplified to:

$$\pi^\top = \pi^\top\left((1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{e}\mathbf{r}^\top\right), \tag{12}$$

which fits the first-order Markov Chain Equation. Here, $\pi^\top$ refers to the stationary distribution, and we denote $\mathbf{P} = (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{e}\mathbf{r}^\top$ as the transition probability matrix, leading to Theorem 1. Due to space limitations, the proof is deferred to the Appendix.

Theorem 1. Let $\mathbf{e}$ denote the column vector of ones, $\mathbf{P} = (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{e}\mathbf{r}^\top$ is the transition probability matrix of the Markov Chain.

3.2. Markov-Chain-based VNG Algorithm

In Markov-chain-based settings, a commonly used algorithm is the Iterative Aggregation (IA) method [17], which is designed to handle nearly decoupled Markov chains. Let $\boldsymbol{\theta}^\top = (\theta_1, \theta_2, \ldots, \theta_m)$ denote the stationary distribution vector for a homogeneous irreducible Markov chain with m states and $\mathbf{Q} \in \mathcal{R}^{m\times m}$ refers to the transition probability matrix. Let $\boldsymbol{\pi}^\top = (\pi_1, \pi_2, \ldots, \pi_n)$ represent the stationary distribution vector for the updated transition probability matrix $\mathbf{P} \in \mathcal{R}^{n\times n}$. The iterative aggregation update process relies on the previously obtained distribution $\boldsymbol{\theta}^\top$ and the updated transition matrix $\mathbf{P}$ to construct a smaller aggregated matrix $\mathbf{U}$. The stationary distribution $\boldsymbol{\phi}^\top$ of $\mathbf{U}$ is then used to derive the updated distribution $\boldsymbol{\pi}^\top$.

Algorithm 1 Markov-Chain-based VNG Algorithm

Input: The adjacency matrix with self-loops $\tilde{\mathbf{A}}$ for G_t
The computed node representation vector $\boldsymbol{\theta}^\top$ for G_{t-1}
The hidden node feature vector $\mathbf{r}^\top$
Output: The updated node representation $\boldsymbol{\pi}^\top$ for G_t

1: Partition the nodes of graph G_t into $S \cup \bar{S}$
2: Let $\tilde{\mathrm{D}}(i,i) = \Sigma_j\tilde{\mathrm{A}}(i,j)$
3: Let $\hat{\tilde{\mathrm{A}}} = \tilde{\mathrm{A}}\tilde{\mathrm{D}}^{-1}$
4: Let $\mathrm{P} = (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathrm{e}\mathrm{r}^\top$ to compute $\boldsymbol{\pi}^\top$
5: Reorder and partition P into $\tilde{\mathrm{P}}$ according to $S \cup \bar{S}$
6: Extract $\bar{\boldsymbol{\theta}}^\top$ from $\boldsymbol{\theta}^\top$ according to $\bar{S}$
7: Let $\tilde{\boldsymbol{s}}^\top = \bar{\boldsymbol{\theta}}^\top/(\bar{\boldsymbol{\theta}}^\top\boldsymbol{e})$
8: Create the aggregated matrix $\tilde{\mathrm{U}}$ by Equation (14)
9: Compute $\boldsymbol{\phi}^\top = (\boldsymbol{\phi}_1, \cdots, \boldsymbol{\phi}_g, \boldsymbol{\phi}_{g+1})$ for $\tilde{\mathrm{U}}$ by Equation (15)
10: Let $\boldsymbol{\pi}^\top = (\boldsymbol{\phi}_1, \cdots, \boldsymbol{\phi}_g \mid \boldsymbol{\phi}_{g+1}\tilde{\boldsymbol{s}}^\top)$
11: Compute $\hat{\boldsymbol{\pi}}^\top = \boldsymbol{\pi}^\top\tilde{\mathrm{P}}$
12: **while** $\mathrm{err}(\hat{\boldsymbol{\pi}}^\top, \boldsymbol{\pi}^\top) > \epsilon$ for a given tolerance ϵ **do**
13: Replace $\boldsymbol{\theta}^\top$ with $\hat{\boldsymbol{\pi}}^\top$
14: Redo lines 06-11
15: **end while**

Without loss of generality, we treat the previously computed node representation vector for graph G_{t-1} as the original stationary distribution vector $\boldsymbol{\theta}^\top$, and the updated node representation vector for the evolved graph G_t as the new stationary distribution vector $\boldsymbol{\pi}^\top$. The core idea behind applying iterative aggregation for updating is to use the previously computed stationary distribution $\boldsymbol{\theta}^\top$, combined with the updated transition probabilities in the matrix $\mathbf{P} = (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{e}\mathbf{r}^\top$, to compute the updated distribution $\boldsymbol{\pi}^\top$. Next, we provide a detailed description of the proposed Markov-chain-based VNG algorithm. The full algorithm for incremental node representation updating is outlined in Algorithm 1.

The states of the updated chain are partitioned into two sets, S and $\bar{S}$. The set S includes the states affected by the update, which consists of both newly added states and the original states modified by the update. The set $\bar{S}$ contains all other unaffected states. Assuming that the number of nodes in the graph updates from m to n, the updated transition matrix $\mathbf{P}$ is reordered and partitioned into $\tilde{\mathbf{P}}$ based on the sets S and $\bar{S}$:

$$\tilde{\mathbf{P}}_{n\times n} = \begin{bmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12} \\ \tilde{\mathbf{P}}_{21} & \tilde{\mathbf{P}}_{22} \end{bmatrix}, \tag{13}$$

where $\tilde{\mathbf{P}}_{11}$ is a block matrix of dimensions $g \times g$ that organizes the transition probabilities between states within S. In the block matrices $\tilde{\mathbf{P}}_{12}$, $\tilde{\mathbf{P}}_{21}$ and $\tilde{\mathbf{P}}_{22}$, the remaining transition probabilities are arranged accordingly.

To further simplify the computation, we group the states in $\bar{S}$ into a single superstate, enabling the construction of a smaller, aggregated transition matrix $\tilde{\mathbf{U}}$, defined as follows:

$$\begin{aligned}\tilde{\mathbf{U}}_{(g+1)\times(g+1)} &= \begin{bmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12}\boldsymbol{e} \\ \tilde{\boldsymbol{s}}^{\top}\tilde{\mathbf{P}}_{21} & \tilde{\boldsymbol{s}}^{\top}\tilde{\mathbf{P}}_{22}\boldsymbol{e} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{I} & 0 \\ 0 & \tilde{\boldsymbol{s}}^{\top} \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12} \\ \tilde{\mathbf{P}}_{21} & \tilde{\mathbf{P}}_{22} \end{bmatrix} \begin{bmatrix} \mathbf{I} & 0 \\ 0 & e \end{bmatrix},\end{aligned} \tag{14}$$

where $\tilde{\boldsymbol{s}}^{\top} = \bar{\boldsymbol{\theta}}^{\top}/(\bar{\boldsymbol{\theta}}^{\top}\boldsymbol{e})$, with $\bar{\boldsymbol{\theta}}^{\top}$ representing the components of $\boldsymbol{\theta}^{\top}$ corresponding to the states in $\bar{S}$. Consequently, the transition matrix $\tilde{\mathbf{P}}$ is aggregated into a $(g+1)\times(g+1)$ matrix, denoted as $\tilde{\mathbf{U}}$.

Next, we compute the stationary distribution $\boldsymbol{\phi}^{\top} = (\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \cdots, \boldsymbol{\phi}_g, \boldsymbol{\phi}_{g+1})$ for the aggregated matrix $\tilde{\mathbf{U}}$ by:

$$(\boldsymbol{\phi}^{(k)})^{\top} = (\boldsymbol{\phi}^{(k-1)})^{\top}\tilde{\mathbf{U}}. \tag{15}$$

Note that to execute Equation (15), a new nested iterative process (local iteration) is required. Therefore, we use index k instead of ℓ to eliminate potential ambiguity.

Let $\boldsymbol{\pi}^{\top} = (\boldsymbol{\phi}_1, \boldsymbol{\phi}_2, \cdots, \boldsymbol{\phi}_g \mid \boldsymbol{\phi}_{g+1}\tilde{\boldsymbol{s}}^{\top})$, and we have $\hat{\boldsymbol{\pi}}^{\top} = \boldsymbol{\pi}^{\top}\tilde{\mathbf{P}}$. Given a relative error threshold ϵ, if $err(\hat{\boldsymbol{\pi}}^{\top} - \boldsymbol{\pi}^{\top}) < \epsilon$, the process terminates. Otherwise, the update continues by re-evaluating $\tilde{\boldsymbol{s}}^{\top} = \bar{\boldsymbol{\pi}}^{\top}/(\bar{\boldsymbol{\pi}}^{\top}\boldsymbol{e})$, where $\bar{\boldsymbol{\pi}}^{\top}$ represents the components of $\hat{\boldsymbol{\pi}}^{\top}$ corresponding to the states in $\bar{S}$. In practice, the relative error $err(\cdot)$ is defined as the average difference between two vectors, $\boldsymbol{v_1}$ and $\boldsymbol{v_2}$, computed using the Mean Absolute Error (MAE), as detailed in Section B of the Appendix.

Fine-Tuning. The refined node representations that Algorithm 1 produces are then utilized to enhance the predictive accuracy of the model $f(\Theta)$ through a fine-tuning process. This fine-tuning step enhances the model's flexibility for downstream applications and reduces Algorithm 1's dependence on specific parameter settings and iterative results, thereby improving stability in the propagation process.

3.3. Theoretical Justification and Analysis

As discussed above, we propose a Markov-chain-based VNG method (Algorithm 1) to compute the updated node representation vector $\boldsymbol{\pi}^{\top}$ for the evolved graph G_t by leveraging the updated transition probabilities matrix $\mathbf{P}$ and the previously established distribution $\theta^{\top}$. In this section, we provide a formal theoretical analysis of the computation process and results. Due to space constraints, all detailed proofs are deferred to the appendix, and only key insights are presented.

Definition 1 (Stochastic Complement [21])**.** Let $\mathbf{P}$ be an irreducible stochastic matrix with a k-level partition:

$$\mathbf{P} = \begin{bmatrix} \mathbf{P}_{11} & \mathbf{P}_{12} & \dots & \mathbf{P}_{1k} \\ \mathbf{P}_{21} & \mathbf{P}_{22} & \dots & \mathbf{P}_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{P}_{k1} & \mathbf{P}_{k2} & \dots & \mathbf{P}_{kk} \end{bmatrix}, \tag{16}$$

where all diagonal blocks are square. Let $\mathbf{P}_i$ denote the submatrix of $\mathbf{P}$ obtained by removing the i-th row and the i-th column of blocks. Similarly, let $\mathbf{P}_{i*}$ represent the i-th row of blocks with $\mathbf{P}_{ii}$ removed, and $\mathbf{P}_{*i}$ is the i-th column of blocks with $\mathbf{P}_{ii}$ removed. The **stochastic complement** of $\mathbf{P}_{ii}$ in $\mathbf{P}$ is defined as:

$$\mathbf{S}_{ii} = \mathbf{P}_{ii} + \mathbf{P}_{i*}(\mathbf{I} - \mathbf{P}_i)^{-1}\mathbf{P}_{*i}. \tag{17}$$

Theorem 2. Let $\mathbf{P}$ be a partitioned transition probability matrix (Equation (16)) for a Markov chain whose stationary probability distribution is:

$$\pi^{\top} = \pi_1^{\top} \left|\pi_2^{\top}\right| \dots \mid \pi_k^{\top}. \tag{18}$$

where $\pi^{\top}$ is partitioned into k segments according to $\mathbf{P}$. Assuming $\phi^{\top} = (\phi_1, \phi_2, \cdots, \phi_k)$ is the stationary distribution for the aggregated chain defined by the aggregated matrix $\mathbf{U}$ in the form:

$$\mathbf{U} = \begin{bmatrix} \boldsymbol{s}_1^{\top}\mathbf{P}_{11}\mathbf{e} & \boldsymbol{s}_1^{\top}\mathbf{P}_{12}\mathbf{e} & \dots & \boldsymbol{s}_1^{\top}\mathbf{P}_{1k}\mathbf{e} \\ \boldsymbol{s}_2^{\top}\mathbf{P}_{21}\mathbf{e} & \boldsymbol{s}_2^{\top}\mathbf{P}_{22}\mathbf{e} & \dots & \boldsymbol{s}_2^{\top}\mathbf{P}_{2k}\mathbf{e} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{s}_k^{\top}\mathbf{P}_{k1}\mathbf{e} & \boldsymbol{s}_k^{\top}\mathbf{P}_{k2}\mathbf{e} & \dots & \boldsymbol{s}_k^{\top}\mathbf{P}_{kk}\mathbf{e} \end{bmatrix}, \tag{19}$$

where $\boldsymbol{s}_i^{\top} = \frac{\pi_i^{\top}}{\pi_i^{\top}\mathbf{e}}$ is the stationary distribution vector for the stochastic complement $\mathbf{S}_{ii}$, then $\phi_i = \pi_i^{\top}\mathbf{e}$, and the stationary distribution for $\mathbf{P}$ is:

$$\pi^{\top} = \left(\phi_1 s_1^{\top} \mid \phi_2 s_2^{\top} \mid \cdots \mid \phi_k s_k^{\top}\right). \tag{20}$$

This result follows from Theorems 2.2 and 4.1 in [21], which provide a detailed discussion, so we omit the proof.

Theorem 2 presents an exact aggregation method for computing the stationary distribution vector $\pi^{\top}$ by first deriving smaller stationary distribution vectors $\boldsymbol{s}_i^{\top}$ from partitioned decoupled Markov chains, and then recombining them to recover $\pi^{\top}$ for the original, non-partitioned larger Markov chain. Although elegant, this approach is inefficient for computing $\pi^{\top}$ due to the costly matrix inversions involved in the stochastic complements (Equation (17)), which are necessary to obtain the stationary distribution $\boldsymbol{s}_i^{\top}$. To improve efficiency, we employ the approximate aggregation approach [17], which bypasses the stochastic complements entirely and directly estimates the distributions $\boldsymbol{s}_i^{\top}$ using the previously known distribution $\theta^{\top}$. Specifically, $\tilde{\boldsymbol{s}}_i^{\top} = \frac{\theta_i^{\top}}{\theta_i^{\top}\mathbf{e}}$. Note that although $\tilde{\boldsymbol{s}}_i^{\top}$ (instead of the exact $\boldsymbol{s}_i^{\top}$) is used, it does not affect the accuracy of $\pi^{\top}$, since the final accuracy of $\pi^{\top}$ is independent of the initial value of $\boldsymbol{s}_i^{\top}$. We will prove this next in Theorem 3. Before doing so, we first present a lemma.

Lemma 1. The exact aggregated transition matrix associated with the partition of $\tilde{\mathbf{P}}$ in Equation (13) is:

$$\mathbf{U}_{(g+1)\times(g+1)} = \begin{bmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12}\boldsymbol{e} \\ \boldsymbol{s}^{\top}\tilde{\mathbf{P}}_{21} & \boldsymbol{s}^{\top}\tilde{\mathbf{P}}_{22}\boldsymbol{e} \end{bmatrix}, \tag{21}$$

where $\boldsymbol{s}^\top$ is the stationary distribution of the stochastic complement:

$$\mathbf{S}_{22} = \tilde{\mathbf{P}}_{22} + \tilde{\mathbf{P}}_{21}(\mathbf{I} - \tilde{\mathbf{P}}_{11})^{-1}\tilde{\mathbf{P}}_{12}. \tag{22}$$

The proof details can be found in Section C.2 of the Appendix. By comparing the approximate aggregated matrix ((14)) used in Algorithm 1 with the exact aggregated matrix (Equation (21)), we observe that the only difference lies in the substitution of $\tilde{\boldsymbol{s}}_i^\top$ for $\boldsymbol{s}_i^\top$. We have Theorem 3 to establish that this substitution does not compromise the accuracy of the final result for $\pi^\top$.

Theorem 3. The proposed Markov-chain-based VNG Algorithm converges to the stationary distribution $\pi^\top$ of $\tilde{\mathbf{P}}$, which is defined in Equation (13).

The details of the proof can be found in Section C.3 of the Appendix. Theorem 3 demonstrates that substituting $\boldsymbol{s}_i^\top$ with $\tilde{\boldsymbol{s}}_i^\top$ does not compromise the accuracy of the final result for $\pi^\top$. As shown by [15], the convergence rate of the iterative aggregation method is at least as fast as that of APPNP (the Power Method). Experimental results in Section 4 further confirm that the proposed VNG method outperforms existing baseline approaches.

4. Experiments

4.1. Experimental Setup

Datasets. To evaluate the effectiveness of our proposed VNG, we follow the setup of APPNP [11] and conduct experiments on four benchmark node-classification datasets: Cora_ML [20], Citeseer [24], PubMed [23], and Microsoft Academic (MS_Academic) [25]. The dataset statistics are summarized in Table 1. Following [11], we partition each dataset into three subsets: a training set, an early-stopping set, and a test set. In the training set, each class has 20 labeled nodes. The datasets provide static graphs. However, for each dataset, by sequentially masking portions of the nodes, we generate sequences of progressively reduced subgraphs. Reversing the sequence effectively simulates the process of a graph with node increment updates.

Baselines. We compare the proposed method with several state-of-the-art methods, including GCN [16], SGC [27], PPNP and APPNP [11]. GCN is the classic model outlined in Section 2.1. SGC is the simplified version of GCN that removes non-linearities, PPNP and APPNP are the PPR-based DGCNs, as discussed in Section 2.2. These models are designed for static graphs, so we retrain them from scratch at each graph snapshot to handle the dynamics of evolving graphs.

In our experiments, we use APPNP [11] as the base model. Initially, node representations are obtained using APPNP, and VNG is then applied to update the representations with each node increment. More details and parameter settings can be found in Section D of the Appendix.

Table 1. Dataset Statistics

Dataset	Nodes	Edges	Features	Classes
Cora_ML	2,810	7,981	2,879	7
Citeseer	2,110	3,668	3,703	6
Pubmed	19,717	44,324	500	3
MS_Academic	18,333	81,894	6,805	15

4.2. Effectiveness and Efficiency Comparison

We run VNG and the baseline models on the datasets, incrementally adding 50 nodes in each step, for a total of 10 increments. For VNG, PPNP, and APPNP, α is set to 0.1. The result is shown in Table 2 and Figure 1 and 2. In Table 2, the results from all snapshots are aggregated to compute the means and standard deviations. In Figure 1 and 2, the details of each snapshot are presented separately. Each data point represents the average result of multiple repetitions of the same experiment. It should be noted that the results from the first iteration are excluded when calculating the means and standard deviations in Table 2, as the initial iteration of VNG essentially corresponds to APPNP.

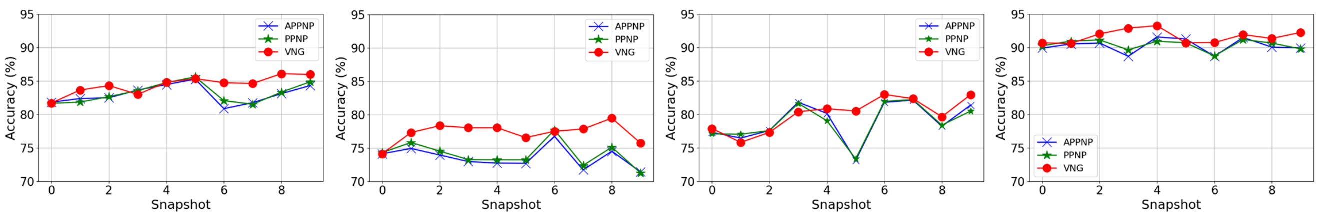
Table 2 demonstrates that on Cora_ML, Pubmed, and MS_Academic, VNG achieves significantly shorter runtimes compared to PPNP and APPNP, while yielding comparable or even superior accuracy. For instance, on the MS_Academic dataset, VNG achieves an average runtime of 27.30 seconds, which is 13.01 seconds faster than the best-performing baseline (APPNP), while simultaneously attaining the highest accuracy of 91.75%. This indicates that VNG is both efficient and effective, as expected and previously inferred. However, on the Citeseer dataset, VNG is slower than PPNP and APPNP, while achieving notably higher accuracy. An intuitive explanation for this is that the Citeseer graph is relatively small (as shown in Table 1), which limits the advantage of VNG's incremental computation, as training on the complete graph is sufficiently fast. The higher accuracy is achieved through the fine-tuning process, which also accounts for the longer runtime.

In Table 2, GCN and SGC exhibit the shortest runtimes; however, their accuracies are relatively low compared to the PPR-based models, rendering them less informative for comparative analysis. Therefore, we will not compare them later.

In Figure 1 and 2, the details of each snapshot are displayed. Overall, VNG demonstrates higher accuracies and shorter runtimes at each snapshot than PPNP and APPNP within an acceptable tolerance range on Cora_ML, Pubmed, and MS_Academic. Also, it can be observed from Figure 2 that the runtime of VNG is not stable, although generally faster. This variability is attributed to the random masking of nodes from the complete graph, resulting in an uneven distribution of nodes connected to the incremented nodes at each snapshot, which in turn leads to differing runtimes. This

Table 2. Effectiveness and Efficiency Comparison

Models	Cora_ML		Citeseer		Pubmed		MS_Acadimic	
	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)
GCN	77.26 ± 2.69	0.17 ± 0.10	65.24 ± 2.69	0.29 ± 0.86	74.59 ± 2.71	0.32 ± 0.08	75.86 ± 4.05	1.57 ± 0.58
SGC	77.66 ± 2.43	0.05 ± 0.01	64.27 ± 2.63	0.05 ± 0.01	74.01 ± 3.12	0.16 ± 0.02	76.72 ± 3.11	0.44 ± 0.05
PPNP	83.35 ± 1.45	9.06 ± 2.36	74.06 ± 2.02	5.26 ± 0.91	79.07 ± 2.77	58.03 ± 9.66	90.42 ± 0.90	117.35 ± 19.03
APPNP	83.00 ± 1.42	19.62 ± 4.74	73.57 ± 1.64	11.75 ± 2.20	79.00 ± 2.91	10.81 ± 2.59	90.28 ± 1.03	40.31 ± 27.26
VNG	84.72 ± 1.19	7.53 ± 7.33	77.64 ± 1.09	13.31 ± 2.02	80.31 ± 2.55	6.39 ± 1.65	91.75 ± 0.12	27.30 ± 7.76



(a) snapshot -Cora_ML (b) snapshot -Citeseer (c) snapshot -Pubmed (d) snapshot -MS_Academic

Figure 1. Comparison of Accuracy on Different Datasets

phenomenon is particularly pronounced on smaller graphs such as Cora_ML and Citeseer, while it is less evident on larger graphs like Pubmed and MS_Academic.

4.3. Node Increment Size Sensitivity

In this section, we conduct experiments using various node increment sizes to facilitate a more thorough analysis of the proposed model. For these experiments, only the Pubmed dataset and the APPNP baseline are used. Except for the parameter under investigation, all other parameters are held constant.

We further set the node increment sizes to 100, 150, and 200, corresponding to 0.5%, 0.75%, and 1% of the complete graph of Pubmed, respectively (50 nodes represent 0.25% of the complete graph). The results are presented in Table 3 and Figure 3, where **N** denotes the node increment size.

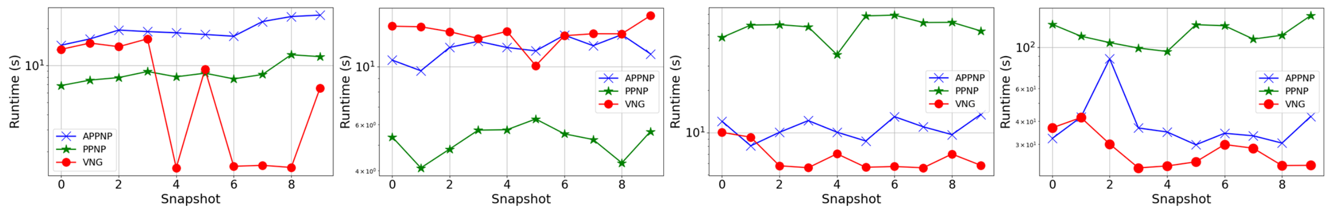
From Table 3, it can be observed that for APPNP, accuracies and runtimes remain nearly constant as **N** changes, as the model's performance is unaffected by this small variation in node count. In contrast, VNG exhibits slight sensitivity to changes in **N**. As shown in Table 3 and Figure 3, the accuracies of VNG remain stable, while the runtimes increase as **N** grows. Specifically, VNG's runtime rises more noticeably when **N** increases from 100 to 150, and when **N** reaches 200 (0.1% of the complete graph), its runtime is comparable to that of APPNP. This indicates that VNG is suitable for smaller increments in node number. It is important to note that, while retraining the model is efficient when the graph undergoes large-scale updates, updating the model remains necessary even for slight increments. This is because the model trained on the original graph cannot be directly applied to the updated graph, as the newly added nodes lack representations.

5. Related Work

A substantial number of node representation learning techniques have been developed to enhance the understanding of graph data. Below, we provide a brief overview of the studies most relevant to the present work.

Graph Convolutional Networks. Motivated by the success of convolutional neural networks (CNNs) for images and text, various graph convolutional networks (GCNs) have been developed to handle the irregular structures of graph data. Bruna et al. [3] first proposed spectral graph convolution using the graph Laplacian matrix. Defferrard et al. [7] subsequently introduced ChebyNets, which employ Chebyshev polynomials to define graph convolutions, avoiding the costly computation of graph Laplacian eigenvectors. Kipf et al. [16] further simplified graph convolutions by stacking layers of first-order Chebyshev polynomial filters. GraphSAGE [13] proposed aggregating information from a sampled subset of neighbors to scale GCNs to large graphs. FastGCN [4] extended GCN scalability by sampling a fixed number of nodes at each layer. GAT [26] incorporated an attention mechanism to assign different aggregation weights to each neighbor. ClusterGCN [6] and GraphSAINT [30] leveraged graph clustering and subgraph structures to reduce the size of the aggregation neighborhood. Here, we list a few representative works; additional GCN methods are discussed in recent surveys [28, 31].

The GCN models discussed thus far handle feature transformation and neighborhood aggregation in a coupled manner, meaning each graph convolution layer comprises both processes. Recent research has identified several issues with this coupled approach, including difficulties in training, challenges in fully leveraging the graph structure, and problems with over-smoothing [11, 27].



(a) snapshot -Cora_ML (b) snapshot -Citeseer (c) snapshot -Pubmed (d) snapshot -MS_Academic

Figure 2. Comparison of Runtime on Different Datasets

Table 3. Node Increment Size Sensitivity

Models	N = 50		N = 100		N = 150		N = 200	
	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)	Accuracy (%)	Runtime (s)
APPNP	79.00 ± 2.91	10.81 ± 2.59	78.88 ± 2.83	10.01 ± 2.65	79.25 ± 2.35	11.58 ± 2.74	78.28 ± 2.32	10.31 ± 2.57
VNG	80.31 ± 2.55	6.39 ± 1.65	82.47 ± 1.98	6.50 ± 2.27	82.05 ± 2.51	7.71 ± 1.80	80.80 ± 1.89	10.48 ± 4.36

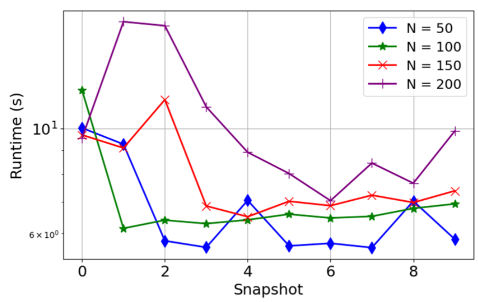


Figure 3. Snapshot runtime of VNG w.r.t. N

Decoupled Graph Convolutional Networks. SGC [27] addresses the training efficiency issues in coupled GCNs by eliminating nonlinearity between GCN layers, thereby decoupling feature transformation and neighborhood aggregation operations. To further mitigate the over-smoothing issue, PPNP and APPNP [11] draw inspiration from Personalized PageRank (PPR) by introducing a residual connection to the initial layer in decoupled GCNs. LightGCN [14] empirically demonstrates that the simplified decoupled GCN architecture outperforms traditional coupled GCNs in both accuracy and efficiency. PPRGo [2] pre-computes an approximate PPR matrix, using it to transform the feature matrix into a final propagation matrix. Similarly, GDC [12] and GBP [5] employ heat kernel PageRank and PPR to capture information from multi-hop neighborhoods, transforming the initial feature matrix into a representation matrix for downstream tasks. A unified optimization framework proposed in [34] illustrates that a well-designed message propagation mechanism can substantially enhance GNN performance. Moreover, Hande Dong et al. [8] prove that decoupled GCNs are theoretically equivalent to label propagation methods. However, a fundamental limitation of these decoupled GCN methods is their reliance on fixed and static graphs.

Recently, several decoupled GCN models have been proposed to address the challenges of dynamically evolving graphs [10, 32, 33]. For example, SDG [10] introduces a simplified dynamic graph neural network that replaces the conventional message-passing mechanism with a dynamic propagation scheme based on tracking Personalized PageRank. InstantGCN [32] presents an incremental computation method for updating the graph representation matrix in dynamic graphs. Furthermore, Zheng et al. [33] propose a unified dynamic propagation framework that enables efficient computation for both continuous and discrete dynamic graphs.

Limitations of Prior Works. To the best of our knowledge, the problem defined in Section 2.3 remains unaddressed in the current literature. Existing dynamic decoupled GCNs are inadequate for this problem, as they are designed to handle only updates to edges and features—a scenario referred to as the *fixed-node evolving* problem. Consequently, they do not account for changes in the number of nodes, also known as the *variable-node evolving* problem. In other words, these models assume a constant number of nodes in the graph, an assumption that is often impractical in real-world applications.

6. Conclusion

This paper proposes a novel decoupled GCN framework, named VNG, which is capable of addressing node-level dynamics. With the core insight that the iterative computation of information propagation in decoupled GCNs can be reformulated as a homogeneous first-order Markov chain, we propose to adapt the iterative aggregation techniques for uncoupling Markov chains to update the previously learned graph node representations incrementally. We provide a theoretical guarantee for our method and demonstrate its efficiency and effectiveness on real-world datasets.

References

[1] Muhammet Balcilar, Guillaume Renton, Pierre Héroux, Benoit Gaüzère, Sébastien Adam, and Paul Honeine. Analyzing the expressive power of graph neural networks in a spectral perspective. In *International Conference on Learning Representations*, 2021. 1

[2] Aleksandar Bojchevski, Johannes Gasteiger, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rózemberczki, Michal Lukasik, and Stephan Günnemann. Scaling graph neural networks with approximate pagerank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, page 2464–2473, New York, NY, USA, 2020. Association for Computing Machinery. 8

[3] J. Bruna, W. Zaremba, A. Szlam, and Y. Lecun. Spectral networks and locally connected networks on graphs. pages 1–10, Banff, Canada, 2014. International Conference on Learning Representations. 1, 7

[4] Jie Chen, Tengfei Ma, and Cao Xiao. FastGCN: Fast learning with graph convolutional networks via importance sampling. In *International Conference on Learning Representations*, 2018. 1, 7

[5] Ming Chen, Zhewei Wei, Bolin Ding, Yaliang Li, Ye Yuan, Xiaoyong Du, and Ji-Rong Wen. Scalable graph neural networks via bidirectional propagation. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2020. Curran Associates Inc. 8

[6] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks. page 257–266, New York, NY, USA, 2019. Association for Computing Machinery. 7

[7] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. page 3844–3852, Red Hook, NY, USA, 2016. Curran Associates Inc. 1, 7

[8] Hande Dong, Jiawei Chen, Fuli Feng, Xiangnan He, Shuxian Bi, Zhaolin Ding, and Peng Cui. On the equivalence of decoupled graph convolution network and label propagation. In *Proceedings of the Web Conference 2021*, page 3651–3662, New York, NY, USA, 2021. Association for Computing Machinery. 8

[9] David Duvenaud, Dougal Maclaurin, Jorge Aguilera-Iparraguirre, Rafael Gómez-Bombarelli, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P. Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, page 2224–2232, Cambridge, MA, USA, 2015. MIT Press. 1

[10] Dongqi Fu and Jingrui He. Sdg: A simplified and dynamic graph neural network. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 2273–2277, New York, NY, USA, 2021. Association for Computing Machinery. 2, 8

[11] Johannes Gasteiger, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Graph neural networks meet personalized pagerank. In *International Conference on Learning Representations*, 2019. 1, 3, 6, 7, 8

[12] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. *Diffusion improves graph learning*. Curran Associates Inc., Red Hook, NY, USA, 2019. 8

[13] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. page 1025–1035, Red Hook, NY, USA, 2017. Curran Associates Inc. 1, 7

[14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, YongDong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 639–648, New York, NY, USA, 2020. Association for Computing Machinery. 1, 8

[15] Ilse C. F. Ipsen and Steve Kirkland. Convergence analysis of a pagerank updating algorithm by langville and meyer. *SIAM Journal on Matrix Analysis and Applications*, 27(4):952–967, 2006. 6

[16] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017. 1, 6, 7

[17] Amy N Langville and Carl D Meyer. Updating markov chains with an eye on google's pagerank. *SIAM journal on matrix analysis and applications*, 27(4):968–987, 2006. 4, 5, 11

[18] Jia Li, Yu Rong, Hong Cheng, Helen Meng, Wenbing Huang, and Junzhou Huang. Semi-supervised graph classification: A hierarchical graph perspective. In *The World Wide Web Conference*, page 972–982, New York, NY, USA, 2019. Association for Computing Machinery. 1

[19] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. AAAI Press, 2018. 2

[20] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, and Kristie Seymore. Automating the construction of internet portals with machine learning. *Inf. Retr.*, 3(2):127–163, 2000. 6, 12

[21] Carl D. Meyer. Stochastic complementation, uncoupling markov chains, and the theory of nearly reducible systems. *SIAM Review*, 31(2):240–272, 1989. 5

[22] Carl D. Meyer, Jr. The role of the group generalized inverse in the theory of finite markov chains. *SIAM Review*, 17(3): 443–464, 1975. 11

[23] Galileo Mark Namata, Ben London, Lise Getoor, and Bert Huang. Query-driven active surveying for collective classification. In *Workshop on Mining and Learning with Graphs*, 2012. 6, 12

[24] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Mag.*, 29(3):93–106, 2008. 6, 12

[25] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. *Relational Representation Learning Workshop*, abs/1811.05868, 2018. 6, 12

[26] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2018. accepted as poster. 1, 7

[27] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *Proceedings of the 36th International Conference on Machine Learning*, pages 6861–6871. PMLR, 2019. 1, 2, 3, 6, 7, 8

[28] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32:4–24, 2019. 7

[29] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. pages 5449–5458. PMLR, 2018. 2

[30] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. GraphSAINT: Graph sampling based inductive learning method. In *International Conference on Learning Representations*, 2020. 7

[31] Ziwei Zhang, Peng Cui, and Wenwu Zhu. Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):249–270, 2022. 7

[32] Yanping Zheng, Hanzhi Wang, Zhewei Wei, Jiajun Liu, and Sibo Wang. Instant graph neural networks for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, page 2605–2615, New York, NY, USA, 2022. Association for Computing Machinery. 2, 8

[33] Yanping Zheng, Zhewei Wei, and Jiajun Liu. Decoupled graph neural networks for large dynamic graphs. *Proc. VLDB Endow.*, 16(9):2239–2247, 2023. 2, 8

[34] Meiqi Zhu, Xiao Wang, Chuan Shi, Houye Ji, and Peng Cui. Interpreting and unifying graph neural networks with an optimization framework. In *Proceedings of the Web Conference 2021*, page 1215–1226, New York, NY, USA, 2021. Association for Computing Machinery. 8

A. Table of Detailed notations

Detailed notations are shown in Table 4.

Table 4. Symbols

Notation	Description
$G = (V, E)$	undirected graph with vertex set V, edge set E
n, m	numbers of nodes and edges in G
X, Y	node feature matrix and the class matrix
$A, \tilde{A}$	G's adjacency matrix without or with self-loops
$A^\top, A^{-1}$	transpose and inverse of matrix A
$D, \tilde{D}$	degree matrix of A and $\tilde{A}$
f, c	number of features and classes
$N_u, \tilde{N}_u$	neighbor set of node u without or with itself
ϵ	relative error threshold
N	node increment size per update

B. Mean Absolute Error

The Mean Absolute Error (MAE) between two vectors v_1 and v_2 is defined as:

$$\mathrm{MAE}(\boldsymbol{v}_1, \boldsymbol{v}_2) = \frac{1}{n}\sum_{k=1}^{n} |\boldsymbol{v}_1(k) - \boldsymbol{v}_2(k)|. \tag{A.1}$$

C. Proofs

C.1. The Proof of Theorem 1

Proof. By the property of the Markov Chain, a finite Markov Chain defined by a stochastic matrix $\mathbf{P}$ has a unique stationary probability distribution if $\mathbf{P}$ is irreducible and aperiodic. We first prove $\mathbf{P}$ is a stochastic, irreducible, and aperiodic matrix, and then prove it is the transition probability matrix of the Markov Chain presented in Equation (10).

Recall that $\mathbf{r}$ is a hidden node feature transformed from the input node feature through a model-agnostic neural network $f_\Theta(.)$. At the final step of this transformation, we augment $\mathbf{r}$ with a very small value to ensure that every element is greater than 0 and normalize it to form a probability vector. Thus we can have $\mathbf{Pe} = (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top \mathbf{e} + \alpha \mathbf{e}\mathbf{r}^\top \mathbf{e} = \mathbf{e}$. Consequently, $\mathbf{P}$ must be stochastic. Since $\mathbf{r}^\top$ is a probability vector with positive elements, making every node of $\mathbf{P}$ directly connecting to every other node, thus $\mathbf{P}$ is irreducible and aperiodic.

Now we prove $\mathbf{P}$ is the transition probability matrix of the Markov Chain presented in Equation (10). Since $\pi^\top$ is the stationary distribution, and thus $\pi^\top \mathbf{e} = 1$. Because $\pi^\top \mathbf{P} = \pi^\top (1-\alpha)\hat{\tilde{\mathrm{A}}}^\top + \pi^\top \alpha \mathbf{e}\mathbf{r}^\top = (1-\alpha)\pi^\top \hat{\tilde{\mathrm{A}}}^\top + \alpha\pi^\top \mathbf{e}\mathbf{r}^\top = (1-\alpha)\pi^\top \hat{\tilde{\mathrm{A}}}^\top + \alpha\mathbf{r}^\top$. Obviously according to Equation (10) we have $\pi^\top = \pi^\top \mathbf{P}$, and thus $\mathbf{P}$ is the transition probability matrix of the Markov Chain presented in Equation (10). □

C.2. The Proof of Lemma 1

Proof. The partition of $\tilde{\mathbf{P}}$ in Equation (13) can be viewed as a $(g+1)$-level partition:

$$\tilde{\mathbf{P}} = \begin{bmatrix} \tilde{p}_{11} & \dots & \tilde{p}_{1g} & \tilde{\mathbf{P}}_{1*} \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{p}_{g1} & \dots & \tilde{p}_{gg} & \tilde{\mathbf{P}}_{g*} \\ \tilde{\mathbf{P}}_{*1} & \dots & \tilde{\mathbf{P}}_{*g} & \tilde{\mathbf{P}}_{22} \end{bmatrix}, \tag{A.2}$$

where

$$\tilde{\mathbf{P}}_{12} = \begin{bmatrix} \tilde{\mathbf{P}}_{1*} \\ \vdots \\ \tilde{\mathbf{P}}_{g*} \end{bmatrix}, \quad \tilde{\mathbf{P}}_{21} = \begin{bmatrix} \tilde{\mathbf{P}}_{*1} & \cdots & \tilde{\mathbf{P}}_{*g} \end{bmatrix}.$$

According to Theorem 2, the exact aggregated matrix of the partition of $\tilde{\mathbf{P}}$ in Equation (13) is:

$$\mathbf{U} = \begin{bmatrix} \mathbf{s}_1^\top \tilde{p}_{11}\mathbf{e} & \dots & \mathbf{s}_1^\top \tilde{p}_{1g}\mathbf{e} & \mathbf{s}_1^\top \tilde{\mathbf{P}}_{1*}\mathbf{e} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{s}_g^\top \tilde{p}_{g1}\mathbf{e} & \dots & \mathbf{s}_g^\top \tilde{p}_{gg}\mathbf{e} & \mathbf{s}_g^\top \tilde{\mathbf{P}}_{g*}\mathbf{e} \\ \mathbf{s}_{g+1}^\top \tilde{\mathbf{P}}_{*1}\mathbf{e} & \dots & \mathbf{s}_{g+1}^\top \tilde{\mathbf{P}}_{*g}\mathbf{e} & \mathbf{s}_{g+1}^\top \tilde{\mathbf{P}}_{22}\mathbf{e} \end{bmatrix}, \tag{A.3}$$

since the first g diagonal blocks have size 1×1 and their corresponding stochastic complements are $\mathbf{S}_{ii} = 1$, and thus their stationary distributions are $\mathbf{s}_i^\top = 1$ for $1 \le i \le g$. $\mathbf{s}^\top$ is the stationary distribution of the stochastic complement in Equation (22), then $\mathbf{s}_{g+1}^\top = \mathbf{s}^\top$, and thus Equation (A.3) can be simplified as:

$$\mathbf{U}_{(g+1)\times(g+1)} = \begin{bmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12}\mathbf{e} \\ \mathbf{s}^\top \tilde{\mathbf{P}}_{21} & \mathbf{s}^\top \tilde{\mathbf{P}}_{22}\mathbf{e} \end{bmatrix}. \tag{A.4}$$

Consequently, the exact aggregated transition matrix associated with the partition of $\tilde{\mathbf{P}}$ in Equation (13) is in the form of Equation (21). □

C.3. The Proof of Theorem 3

Proof. According to [17], let $\mathbf{s}^{(0)\top}$ be any initial probability vector. Let $\mathbf{s}^{(\ell)\top}$, $\mathbf{U}^{(\ell)}$, $\phi^{(\ell)\top}$, $\boldsymbol{\pi}^{(\ell)\top}$, and $\hat{\boldsymbol{\pi}}^{(\ell)\top}$ be the corresponding results from steps 06-11 after ℓ iterations of Algorithm 1. We have:

$$\mathbf{U}^{(\ell+1)} = \begin{pmatrix} \tilde{\mathbf{P}}_{11} & \tilde{\mathbf{P}}_{12}\mathbf{e} \\ \mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21} & \mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{22}\mathbf{e} \end{pmatrix}.$$

According to group generalized inverse theory of Finite Markov Chains [22], the stationary distribution $\boldsymbol{\phi}^{(\ell+1)\top}$ of

$\mathbf{U}^{(\ell+1)}$ can be computed by:

$$
\begin{aligned}
\boldsymbol{\phi}^{(\ell+1)\top} &= \left(\mathbf{I} - \mathbf{U}^{(\ell+1)}\right)^{-1} \\
&= \begin{pmatrix} \mathbf{I} - \tilde{\mathbf{P}}_{11} & -\tilde{\mathbf{P}}_{12}\mathbf{e} \\ -\mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21} & \mathbf{I} - \mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{22}\mathbf{e} \end{pmatrix}^{-1} \\
&= \frac{1}{\beta^{(\ell+1)}} \left(\mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1} \mid 1\right),
\end{aligned}
$$

in which $\beta^{(\ell+1)} = 1 + \mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1}\mathbf{e}$. Since $\mathbf{s}_i^{(\ell)\top} = 1$ for $1 \leq i \leq g$ and $\mathbf{s}_{g+1}^{(\ell)\top} = \mathbf{s}^{(\ell)\top}$, according to Equation (20), we have:

$$
\begin{aligned}
\boldsymbol{\pi}^{(\ell+1)\top} &= \left(\phi_1^{(\ell+1)}\mathbf{s}_1^{(\ell)\top} \mid \cdots \mid \phi_g^{(\ell+1)}\mathbf{s}_g^{(\ell)\top} \mid \phi_{g+1}^{(\ell+1)}\mathbf{s}_{g+1}^{(\ell)\top}\right) \\
&= \left(\phi_1^{(\ell+1)}, \cdots, \phi_g^{(\ell+1)} \mid \phi_{g+1}^{(\ell+1)}\mathbf{s}^{(\ell)\top}\right) \\
&= \frac{1}{\beta^{(\ell+1)}} \left(\mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1} \mid \mathbf{s}^{(\ell)\top}\right).
\end{aligned}
$$

Since the first g diagonal blocks have size 1×1 and their corresponding stochastic complements are $\mathbf{S}_{ii} = 1$ and $\mathbf{S}_{(g+1)(g+1)} = \mathbf{S}_{22}$ (Equation (22)), we have:

$$
\begin{aligned}
\hat{\boldsymbol{\pi}}^{(\ell+1)\top} &= \boldsymbol{\pi}^{(\ell+1)\top}\tilde{\mathbf{P}} \\
&= \left(\phi_1^{(\ell+1)}\mathbf{S}_{11}, \cdots, \phi_g^{(\ell+1)}\mathbf{S}_{gg} \mid \phi_{g+1}^{(\ell+1)}\mathbf{s}^{(\ell)\top}\mathbf{S}_{(g+1)(g+1)}\right) \\
&= \left(\phi_1^{(\ell+1)}, \cdots, \phi_g^{(\ell+1)} \mid \phi_{g+1}^{(\ell+1)}\mathbf{s}^{(\ell)\top}\mathbf{S}_{22}\right) \\
&= \frac{1}{\beta^{(\ell+1)}} \left(\mathbf{s}^{(\ell)\top}\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1} \mid \mathbf{s}^{(\ell)\top}\mathbf{S}_{22}\right), \\
\mathbf{s}^{(\ell+1)\top} &= \mathbf{s}^{(\ell)\top}\mathbf{S}_{22} = \mathbf{s}^{(0)\top}\mathbf{S}_{22}^{\ell}.
\end{aligned}
$$

This shows that as $\ell \to \infty$, $\mathbf{s}^{(\ell)\top}$ converges to $\mathbf{s}^\top$ (the stationary distribution associated with $\mathbf{S}_{22}$), regardless of the initial value $\mathbf{s}^{(0)\top}$. As $\mathbf{s}^{(\ell)\top} \to \mathbf{s}^\top$, we have

$$\beta^{(\ell)} \to \beta = 1 + \mathbf{s}^\top\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1}\mathbf{e}$$

and

$$\boldsymbol{\pi}^{(\ell)\top} \to \frac{1}{\beta^{(\ell)}}\left(\mathbf{s}^\top\tilde{\mathbf{P}}_{21}\left(\mathbf{I} - \tilde{\mathbf{P}}_{11}\right)^{-1} \mid \mathbf{s}^\top\right) = \boldsymbol{\pi}^\top.$$

□

D. Experiment Details

The source code of VNG is available at: https://anonymous.4open.science/r/VNG-038F.

D.1. Datasets

The datasets utilized in this study include Cora_ML [20], Citeseer [24], PubMed [23], and Microsoft Academic [25]. Cora_ML, Citeseer, and PubMed are citation networks where nodes correspond to academic papers, and edges represent citation relationships. The Microsoft Academic dataset represents a co-authorship network, with nodes denoting authors and edges indicating co-authorship relationships.

D.2. Implementation Details

Model Hyperparameters. To ensure a fair comparison across models, we utilized a neural network architecture for VNG that aligns with PPNP and APPNP, maintaining the same parameter count. We use two layers with $h = 64$ hidden units. We apply L2 regularization with $\lambda = 0.005$ on the weights of the first layer and use dropout with dropout rate $d = 0.5$ on both layers and the adjacency matrix. For VNG and APPNP, adjacency dropout is resampled for each power iteration step. For propagation, we use the teleport probability $\alpha = 0.1$ and $K = 10$ power iteration steps for both VNG and APPNP. The early stopping criterion is configured with a patience value of $p = 100$ and a maximum of $n = 10,000$ epochs. We employ the Adam optimizer with a learning rate of $l = 0.01$ for optimization, using cross-entropy loss as the objective function across all models.

Baseline Hyperparameters. The hyperparameters of PPNP and APPNP are stated above. For GCN, we use three layers, with the hidden layer size being $h = 64$. The dropout rate is $d = 0.0$ (i.e. without dropout), the learning rate is $l = 0.01$, the weight decay is $\lambda = 0.005$, the maximum number of epochs is $n = 200$, and the early stopping patience is $p = 100$. And for SGC, we use two layers with $h = 64$. The learning rate is set to $l = 0.01$, while weight decay is applied with $\lambda = 5 \times 10^{-4}$ to regularize the model. Training is performed for a maximum of $n = 200$, with an early stopping criterion defined by a patience value of $p = 10$. Dropout is applied with a rate of $d = 0.5$. The Adam optimizer is then applied to both GCN and SGC, with a learning rate of $l = 0.01$, and the loss is calculated using the negative log-likelihood (NLL) loss combined with L2 regularization.

We implement VNG in PyTorch. All experiments are completed on a machine with an NVIDIA RTX 4090 GPU (24GB memory), Intel Core™ i9-10900K CPU (3.7 GHz) with 20 cores, and 64GB of RAM.